Importing Libraries and Packages

```
import random
import uuid
from typing import List, Tuple, Dict, Any, Union
import datetime

import numpy as np
import statsmodels.api as sm
from statsmodels.formula.api import ols
import pandas as pd

import xgboost as xgb
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Lasso
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error
```

Dataset Generation

Designing the City

```
CustomerRecord = Dict[str, Union[str, int, float]]
DriverRecord = Dict[str, Union[str, int, float, bool]]
CarRecord = Dict[str, Union[str, int]]
RideRecord = Dict[str, Union[str, int, float]]

#Adding weights to control random data generation
TIME_CALC_WEIGHTS: Dict[str, float] = {
    'trip_distance_units': 1.5,
    'pickup_minute_of_day': 0.005,


    'Commercial_Penalty': 0.7,
    'Road_Bonus': -0.4,
    'Fri_Penalty': 0.5,
    'Sat_Penalty': 0.2,

    'zone_speed_limit_impact': -0.002,

    'BASE_TIME': 2.0,
    'NOISE_STD_DEV': 0.5
}

ZONE_SPEED_LIMITS: Dict[str, float] = {
    'Residential': 25.0,
    'Commercial': 15.0,
    'Road': 45.0,
    'Park': 10.0,
    'Industrial': 30.0,
```

```
    'Water': 5.0,
    'Empty': 20.0
}

class CityGrid:
    """
    Represents an NxN grid-based city map with different zones.
    """

    def __init__(self, size: int):
        if size <= 0:
            raise ValueError("City size N must be a positive
integer.")
        self.size = size
        self.grid: List[List[str]] = [['Empty' for _ in range(size)]
for _ in range(size)]

        self.zone_symbols: Dict[str, str] = {
            'Empty': '  .  ',
            'Residential': '  R  ',
            'Commercial': '  C  ',
            'Park': '  P  ',
            'Industrial': '  I  ',
            'Road': '  #  ',
            'Water': '  ~  ',
        }
        print(f"City Grid initialized: {self.size}x{self.size} size.")


    def _is_valid_coord(self, x: int, y: int) -> bool:
        return 0 <= x < self.size and 0 <= y < self.size


    def place_zone(self, x: int, y: int, zone_name: str) -> bool:
        if not self._is_valid_coord(x, y):
            print(f"Error: Coordinates ({x}, {y}) are outside the
{self.size}x{self.size} grid.")
            return False

        if zone_name not in self.zone_symbols:
            print(f"Warning: Zone '{zone_name}' is not predefined.
Using default symbol.")
            self.zone_symbols[zone_name] = f" {zone_name[0].upper()} "

        self.grid[x][y] = zone_name
        return True


    def get_zone(self, x: int, y: int) -> str | None:
        if not self._is_valid_coord(x, y):
```

```
            return None
        return self.grid[x][y]


    def display(self):
        print("\n" + "=" * (self.size * 5 + 10))
        print(f"CITY MAP ({self.size}x{self.size})")
        print("=" * (self.size * 5 + 10))

        header = "    "
        for i in range(self.size):
            header += f" (y={i:02}) "
        print(header)
        print("  " + "-" * (self.size * 6 + 1))

        for x in range(self.size):
            row_output = f"(x={x:02}) |"
            for y in range(self.size):
                zone = self.grid[x][y]
                symbol = self.zone_symbols.get(zone, ' ??? ')
                row_output += symbol
            row_output += "|"
            print(row_output)

        print("  " + "-" * (self.size * 6 + 1))

        print("\nLegend:")
        for zone, symbol in self.zone_symbols.items():
            print(f"  {symbol.strip()} = {zone}")
        print("-" * (self.size * 5 + 10))


def calculate_distance(p1: Tuple[int, int], p2: Tuple[int, int]) ->
int:
    """Calculates the Manhattan distance (L1 norm) between two grid
points."""
    return abs(p1[0] - p2[0]) + abs(p1[1] - p2[1])
```

Designing the Data Structure

```
def generate_unique_entities(num_entities: int) ->
Tuple[List[CustomerRecord], List[DriverRecord], List[CarRecord]]:
    """Generates lists of unique customer, driver, and car
entities."""

    customers: List[CustomerRecord] = []
    drivers: List[DriverRecord] = []
    cars: List[CarRecord] = []

    # 1. Customer Data
```

```
    for i in range(num_entities):
        customers.append({
            'customer_id': str(uuid.uuid4())[:8],
            'customer_since_year': random.randint(2018, 2024),
            'rating_avg': round(random.uniform(4.0, 5.0), 2)
        })

    # 2. Driver Data
    for i in range(num_entities):
        drivers.append({
            'driver_id': f"DRV-{random.randint(1000, 9999)}",
            'driver_tenure_months': random.randint(3, 72),
            'driver_rating': round(random.uniform(4.5, 5.0), 2),
            'is_full_time': random.choice([True, False])
        })

    # 3. Car Data
    makes = ['Toyota', 'Honda', 'Tesla', 'Ford', 'BMW']
    models = ['Sedan', 'SUV', 'Hatchback', 'Electric']
    for i in range(num_entities):
        cars.append({
            'car_id': f"CAR-{random.randint(100, 999)}",
            'car_make': random.choice(makes),
            'car_model_type': random.choice(models),
            'car_year': random.randint(2019, 2024)
        })

    return customers, drivers, cars


def generate_ride_dataset(
    city: CityGrid,
    num_rides: int,
    customers: List[CustomerRecord],
    drivers: List[DriverRecord],
    cars: List[CarRecord]
) -> List[RideRecord]:
    """
    Generates a sample dataset of ride records using static weights to
enforce correlations.
    """
    rides: List[RideRecord] = []
    days = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]

    print(f"\nGenerating {num_rides} ride records with weighted time
calculation to enforce correlations...")

    customer_ids = [c['customer_id'] for c in customers]
    driver_ids = [d['driver_id'] for d in drivers]
    car_ids = [c['car_id'] for c in cars]
```

```
    for i in range(num_rides):

        customer_id = random.choice(customer_ids)
        driver_id = random.choice(driver_ids)
        car_id = random.choice(car_ids)

        px, py = random.randint(0, city.size - 1), random.randint(0,
city.size - 1)
        dx, dy = random.randint(0, city.size - 1), random.randint(0,
city.size - 1)

        pickup_coord = (px, py)
        dropoff_coord = (dx, dy)

        pickup_zone = city.get_zone(px, py)
        dropoff_zone = city.get_zone(dx, dy)

        ride_speed_limit = ZONE_SPEED_LIMITS.get(pickup_zone,
ZONE_SPEED_LIMITS['Empty'])

        day_of_week = random.choice(days)

        request_hour = random.randint(0, 23)
        request_minute = random.randint(0, 59)
        request_time = f"{request_hour:02}:{request_minute:02}"
        request_minutes_total = request_hour * 60 + request_minute

        base_acceptance_delay = 1.0
        if pickup_zone == 'Commercial':
            acceptance_delay = max(0.2, base_acceptance_delay *
random.uniform(0.5, 0.9))
        else:
            acceptance_delay = base_acceptance_delay *
random.uniform(0.8, 1.5)

        acceptance_minutes_total = request_minutes_total +
acceptance_delay

        driver_to_pickup_distance = random.randint(1, 5)
        time_to_pickup_minutes = driver_to_pickup_distance / 1.0

        pickup_minutes_total = acceptance_minutes_total +
time_to_pickup_minutes
        pickup_minute_of_day = pickup_minutes_total % 1440


        trip_distance_units = calculate_distance(pickup_coord,
dropoff_coord)

        trip_time_minutes_calc = TIME_CALC_WEIGHTS['BASE_TIME']
```

```
        trip_time_minutes_calc += trip_distance_units *
TIME_CALC_WEIGHTS['trip_distance_units']

        trip_time_minutes_calc += pickup_minute_of_day *
TIME_CALC_WEIGHTS['pickup_minute_of_day']


        if pickup_zone == 'Commercial' or dropoff_zone ==
'Commercial':
             trip_time_minutes_calc +=
TIME_CALC_WEIGHTS['Commercial_Penalty']

        if pickup_zone == 'Road' or dropoff_zone == 'Road':
             trip_time_minutes_calc += TIME_CALC_WEIGHTS['Road_Bonus']

        if day_of_week == 'Fri':
            trip_time_minutes_calc += TIME_CALC_WEIGHTS['Fri_Penalty']
        elif day_of_week == 'Sat':
            trip_time_minutes_calc += TIME_CALC_WEIGHTS['Sat_Penalty']


        trip_time_minutes_calc += ride_speed_limit *
TIME_CALC_WEIGHTS['zone_speed_limit_impact']


        noise = np.random.normal(0,
TIME_CALC_WEIGHTS['NOISE_STD_DEV'])
        trip_time_minutes = max(1.0, trip_time_minutes_calc + noise)


        pickup_hour = int(pickup_minutes_total // 60) % 24
        pickup_minute = int(pickup_minutes_total % 60)
        pickup_time = f"{pickup_hour:02}:{pickup_minute:02}"


        dropoff_minutes_total = pickup_minutes_total +
trip_time_minutes
        dropoff_hour = int(dropoff_minutes_total // 60) % 24
        dropoff_minute = int(dropoff_minutes_total % 60)
        dropoff_time = f"{dropoff_hour:02}:{dropoff_minute:02}"


        record: RideRecord = {
            'ride_id': str(uuid.uuid4())[:8],
            'customer_id': customer_id,
            'driver_id': driver_id,
            'car_id': car_id,
            'day_of_week': day_of_week,
            'request_time': request_time,
            'driver_acceptance_time': f"{int(acceptance_minutes_total
```

```
// 60) % 24:02}:{int(acceptance_minutes_total % 60):02}",
            'pickup_time': pickup_time,
            'dropoff_time': dropoff_time,
            'wait_time_minutes': round(acceptance_delay, 2),
            'trip_time_minutes': round(trip_time_minutes, 2),
            'pickup_x': px,
            'pickup_y': py,
            'pickup_zone': pickup_zone,
            'dropoff_x': dx,
            'dropoff_y': dy,
            'dropoff_zone': dropoff_zone,
            'trip_distance_units': trip_distance_units,
            'zone_speed_limit': ride_speed_limit
        }
        rides.append(record)

    return rides


def assign_dates_to_rides(ride_df: pd.DataFrame, start_date_str: str =
'2025-11-01') -> pd.DataFrame:
    """
    Adds a 'ride_date' column to the ride DataFrame.
    It assigns a date based on the 'day_of_week' column, ensuring
consistency.
    """
    day_mapping = {
        'Mon': 0, 'Tue': 1, 'Wed': 2, 'Thu': 3,
        'Fri': 4, 'Sat': 5, 'Sun': 6
    }

    try:
        start_date = datetime.datetime.strptime(start_date_str, '%Y-
%m-%d').date()
    except ValueError:
        print(f"Warning: Invalid start date format '{start_date_str}'.
Using today's date.")
        start_date = datetime.date.today()

    target_dates: Dict[str, List[datetime.date]] = {day: [] for day in
day_mapping.keys()}

    for i in range(30):
        current_date = start_date + datetime.timedelta(days=i)
        current_day_str = current_date.strftime('%a')[:3]
        if current_day_str in target_dates:
            target_dates[current_day_str].append(current_date)

    ride_df['ride_date'] = None
```

```
    for day, group in ride_df.groupby('day_of_week'):
        if day in target_dates and target_dates[day]:
            assigned_date = target_dates[day][0]
            ride_df.loc[group.index, 'ride_date'] =
assigned_date.strftime('%Y-%m-%d')
        else:
            ride_df.loc[group.index, 'ride_date'] = '2025-11-01'

    cols = ['ride_id', 'ride_date'] + [col for col in ride_df.columns
if col not in ['ride_id', 'ride_date']]
    return ride_df[cols]

def print_table(df: pd.DataFrame, title: str):
    """Prints a pandas DataFrame as a formatted table."""
    if df.empty:
        print(f"\n--- {title} (0 Records) ---")
        return

    print(f"\n--- {title} ({len(df)} Records) ---")
    print(df.head(10).to_string())
    if len(df) > 10:
        print(f"... showing first 10 of {len(df)} records ...")
    print("-" * (len(title) + 20))
```

Displaying the City and generated datasets

```
if __name__ == "__main__":

    # 1. Create a 10x10 city
    CITY_SIZE = 10
    NUM_ENTITIES = 5
    NUM_RIDES = 15

    my_city = CityGrid(CITY_SIZE)

    # 2. Place specific zones
    print("\nPlacing zones...")

    # Residential Area
    for x in range(1, 4):
        for y in range(1, 4):
            my_city.place_zone(x, y, 'Residential')

    # Commercial Center
    my_city.place_zone(8, 2, 'Commercial')
    my_city.place_zone(8, 3, 'Commercial')
    my_city.place_zone(9, 2, 'Commercial')
    my_city.place_zone(9, 3, 'Commercial')

    # Park
```

```
    for x in range(4, 7):
        my_city.place_zone(x, 7, 'Park')

    # Road network
    print("Adding grid road network for better accessibility...")
    road_indices = [2, 5, 8]

    for r in road_indices:
        for c in range(CITY_SIZE):
            my_city.place_zone(r, c, 'Road')

    for c in road_indices:
        for r in range(CITY_SIZE):
            my_city.place_zone(r, c, 'Road')

    # Water zone
    my_city.place_zone(0, 0, 'Water')
    my_city.place_zone(0, 1, 'Water')
    my_city.place_zone(1, 0, 'Water')

    # 3. Display the fixed city map
    my_city.display()

    # 4. Generate unique entities (as lists of dicts)
    customer_list, driver_list, car_list =
generate_unique_entities(NUM_ENTITIES)

    # 5. Generate ride records (as list of dicts)
    ride_list = generate_ride_dataset(my_city, NUM_RIDES,
customer_list, driver_list, car_list)

    # 6. Convert lists of dictionaries to Pandas DataFrames
    customer_df = pd.DataFrame(customer_list)
    driver_df = pd.DataFrame(driver_list)
    car_df = pd.DataFrame(car_list)
    ride_df = pd.DataFrame(ride_list)

    ride_df = assign_dates_to_rides(ride_df, start_date_str='2025-11-
01')

    dataset = ride_df #Keep for dashboard

    # 7. Print all four DataFrames
    print("\n" + "=" * 80)
    print("RIDE-HAILING SIMULATION DATA TABLES (Pandas DataFrames) -
WITH NEW SPEED LIMITS & DATES")
    print("=" * 80)

    print_table(customer_df, "Customer Information")
    print_table(driver_df, "Driver Information")
```

```
    print_table(car_df, "Car Information")

    print_table(ride_df, "Ride Transaction Records (Links Entities)")
```

```
City Grid initialized: 10x10 size.

Placing zones...
Adding grid road network for better accessibility...

============================================================
CITY MAP (10x10)
============================================================
     (y=00)  (y=01)  (y=02)  (y=03)  (y=04)  (y=05)  (y=06)  (y=07)
(y=08)  (y=09)
  ------------------------------------------------------------
(x=00) |    ~       ~       #       .       .       #       .       .
#       .   |
(x=01) |    ~       R       #       R       .       #       .       .
#       .   |
(x=02) |    #       #       #       #       #       #       #       #
#       #   |
(x=03) |    .       R       #       R       .       #       .       .
#       .   |
(x=04) |    .       .       #       .       .       #       .       P
#       .   |
(x=05) |    #       #       #       #       #       #       #       #
#       #   |
(x=06) |    .       .       #       .       .       #       .       P
#       .   |
(x=07) |    .       .       #       .       .       #       .       .
#       .   |
(x=08) |    #       #       #       #       #       #       #       #
#       #   |
(x=09) |    .       .       #       C       .       #       .       .
#       .   |
  ------------------------------------------------------------

Legend:
  . = Empty
  R = Residential
  C = Commercial
  P = Park
  I = Industrial
  # = Road
  ~ = Water
------------------------------------------------------------

Generating 15 ride records with weighted time calculation to enforce
correlations...
```

```
================================================================================
RIDE-HAILING SIMULATION DATA TABLES (Pandas DataFrames) - WITH NEW SPEED LIMITS & DATES
================================================================================

--- Customer Information (5 Records) ---
  customer_id  customer_since_year  rating_avg
0    7763decd                2024        4.47
1    e4acbe16                2019        4.79
2    70f606bb                2022        4.17
3    c37c6686                2024        4.21
4    b7786e99                2022        4.71
----------------------------------------

--- Driver Information (5 Records) ---
  driver_id  driver_tenure_months  driver_rating  is_full_time
0  DRV-6722                    23           4.95         False
1  DRV-8163                    38           4.75         False
2  DRV-7634                    21           4.68          True
3  DRV-3549                    53           4.72         False
4  DRV-1537                    22           4.56         False
--------------------------------------

--- Car Information (5 Records) ---
    car_id car_make car_model_type  car_year
0  CAR-177    Honda          Sedan      2022
1  CAR-605     Ford            SUV      2023
2  CAR-938    Honda            SUV      2021
3  CAR-491    Honda            SUV      2021
4  CAR-351      BMW          Sedan      2021
----------------------------------

--- Ride Transaction Records (Links Entities) (15 Records) ---
    ride_id   ride_date customer_id driver_id   car_id day_of_week
request_time driver_acceptance_time pickup_time dropoff_time
wait_time_minutes  trip_time_minutes  pickup_x  pickup_y pickup_zone
dropoff_x  dropoff_y dropoff_zone  trip_distance_units
zone_speed_limit
0  9ca53daf  2025-11-04    c37c6686  DRV-1537  CAR-491         Tue
14:23                  14:24       14:27        14:36
1.19               9.76         6         5        Road           6
8         Road                    3              45.0
1  8d3d81ce  2025-11-07    c37c6686  DRV-3549  CAR-491         Fri
07:07                  07:08       07:11        07:24
1.38              12.85         5         5        Road           7
9        Empty                    6              45.0
2  6557dd7d  2025-11-04    70f606bb  DRV-8163  CAR-177         Tue
14:18                  14:19       14:22        14:43
```

```
1.24              21.01         0         5        Road           6
1        Empty                   10              45.0
3  73f59a8e  2025-11-05    c37c6686  DRV-3549  CAR-491         Wed
16:05                  16:05       16:10        16:23
0.87              12.48         7         4        Empty          9
2         Road                    4              20.0
4  344ae507  2025-11-02    7763decd  DRV-8163  CAR-491         Sun
07:25                  07:26       07:28        07:44
1.29              15.90         0         7        Empty          3
2         Road                    8              20.0
5  850484a7  2025-11-03    c37c6686  DRV-8163  CAR-351         Mon
11:58                  11:59       12:04        12:11
1.36               7.53         8         3        Road           7
4        Empty                     2              45.0
6  2dde6065  2025-11-07    c37c6686  DRV-7634  CAR-605         Fri
18:24                  18:25       18:28        18:40
1.36              12.49         1         6        Empty          2
4         Road                    3              20.0
7  373d6dea  2025-11-01    70f606bb  DRV-6722  CAR-491         Sat
00:58                  00:58       00:59        01:13
0.85              13.70         4         9        Empty          0
5         Road                    8              20.0
8  ebb4b728  2025-11-07    b7786e99  DRV-1537  CAR-351         Fri
00:08                  00:08       00:09        00:24
0.90              14.49         3         9        Empty          3
1  Residential                    8              20.0
9  276b7416  2025-11-07    e4acbe16  DRV-3549  CAR-938         Fri
12:22                  12:23       12:27        12:49
1.11              22.78         8         4        Road           1
0        Water                    11              45.0
... showing first 10 of 15 records ...
------------------------------------------------------------
```

Hypothesis Testing on Assumptions The total time taken on a trip is affected by

- The pickup and drop time
- The distance
- The day of the week
- The pickup and drop off zones

```python
#Converting time to minutes
def time_to_minute(time_str):
    """Converts a time string 'HH:MM' to the minute of the day."""
    try:
        h, m = map(int, time_str.split(':'))
        return h * 60 + m
    except:
        return np.nan
```

```
ride_df['pickup_minute_of_day'] =
ride_df['pickup_time'].apply(time_to_minute)
print(f"Created 'pickup_minute_of_day' feature (Min:
{ride_df['pickup_minute_of_day'].min()}, Max:
{ride_df['pickup_minute_of_day'].max()})")

ride_df_clean = ride_df.dropna(subset=['trip_time_minutes',
                                       'trip_distance_units',
                                       'pickup_minute_of_day',
                                       'day_of_week']).copy()

# The model will use:
# - trip_distance_units (continuous)
# - pickup_minute_of_day (continuous)
# - C(day_of_week) (categorical, automatically converted to dummy
variables)
formula = 'trip_time_minutes ~ trip_distance_units +
pickup_minute_of_day + C(day_of_week)'
print(f"Regression Formula: {formula}")

model = ols(formula, data=ride_df_clean).fit()

print("\n--- Regression Results Summary ---")
print(model.summary().as_text())


print("\n" + "=" * 60)
print("--- Interpretation of Regression Results ---")
print("=" * 60)

f_pvalue = model.f_pvalue
r_squared = model.rsquared

print("## 1. Overall Model Significance (F-Test)")
print("-" * 35)

if f_pvalue < 0.05:
    print(f"**Conclusion:** The overall model is **statistically
significant** (p-value: {f_pvalue:.4f}).")
    print("This means the group of tested factors (distance, time of
day, and day of week) is useful in predicting trip duration.")
    print(f"**R-squared:** {r_squared:.4f}. This indicates that
{r_squared*100:.2f}% of the variance in trip time is explained by the
factors in this model.")
else:
    print(f"**Conclusion:** The overall model is NOT statistically
significant (p-value: {f_pvalue:.4f}).")

print("\n" + "*" * 60)
```

```
results_table = model.pvalues.reset_index().rename(columns={'index':
'Feature', 0: 'p_value'})
coef_table = model.params.reset_index().rename(columns={'index':
'Feature', 0: 'Coefficient'})
results_df = pd.merge(results_table, coef_table, on='Feature')

# --- Hypothesis 1: Distance (trip_distance_units) ---
dist_p = results_df[results_df['Feature'] == 'trip_distance_units']
['p_value'].iloc[0]
dist_coef = results_df[results_df['Feature'] == 'trip_distance_units']
['Coefficient'].iloc[0]

print("## 2. Hypothesis Test for Trip Distance (Distance)")
print("-" * 35)
print(f"Null Hypothesis (H0): Distance has no effect on trip time.")

if dist_p < 0.05:
    print(f"**Result:** Reject H0. The effect of distance is
**statistically significant** (p-value: {dist_p:.4f}).")
    print(f"**Effect (Coefficient):** {dist_coef:.4f}")
    print("Interpretation: For every 1-unit increase in distance, the
trip time increases by approximately **{:.2f} minutes**, holding all
other factors constant.".format(dist_coef))
else:
    print(f"**Result:** Fail to Reject H0. The effect of distance is
NOT statistically significant (p-value: {dist_p:.4f}).")

print("\n" + "*" * 60)

# --- Hypothesis 2: Time of Pickup (pickup_minute_of_day) ---
time_p = results_df[results_df['Feature'] == 'pickup_minute_of_day']
['p_value'].iloc[0]
time_coef = results_df[results_df['Feature'] ==
'pickup_minute_of_day']['Coefficient'].iloc[0]

print("## 3. Hypothesis Test for Time of Pickup (Minute of Day)")
print("-" * 35)
print(f"Null Hypothesis (H0): Time of day has no effect on trip
time.")

if time_p < 0.05:
    print(f"**Result:** Reject H0. The effect of time of day is
**statistically significant** (p-value: {time_p:.4f}).")
    print(f"**Effect (Coefficient):** {time_coef:.6f}")
    print("Interpretation: The time of day significantly affects trip
time. The positive coefficient suggests that later times in the day
(higher minute count) are associated with **slightly longer trip
times** (or that the inherent speed in the simulation varies by
time).")
else:
```

```
    print(f"**Result:** Fail to Reject H0. The effect of time of day
is NOT statistically significant (p-value: {time_p:.4f}).")

print("\n" + "*" * 60)

# --- Hypothesis 3: Day of Week (C(day_of_week)) ---
day_of_week_p_values =
results_df[results_df['Feature'].str.contains(r'C\(day_of_week\)\
[T\.', regex=True)]['p_value']
significant_days = day_of_week_p_values[day_of_week_p_values < 0.05]

print("## 4. Hypothesis Test for Day of Week")
print("-" * 35)
print(f"Null Hypothesis (H0): The day of the week has no effect on
trip time (all day coefficients are zero).")

if not significant_days.empty:
    print(f"**Result:** Reject H0. At least one day is **statistically
significant**.")
    print("Days that show a statistically different trip time
(compared to the baseline day, which is excluded by the model):")
    for index, p_value in significant_days.items():
        feature = results_df.loc[index, 'Feature']
        coef = results_df.loc[index, 'Coefficient']
        print(f"  - {feature.split('[T.')[1].replace(']', '')}:
**Significant** (p={p_value:.4f}, Coef={coef:.4f})")
    print("\nInterpretation: The trip time on these specific days is
significantly different from the baseline day, suggesting **varying
levels of congestion/speed limit effects** depending on the day.")
else:
    print("**Result:** Fail to Reject H0. No individual day of the
week is found to be statistically significant from the baseline day.")

print("=" * 60)
```

```
Created 'pickup_minute_of_day' feature (Min: 9, Max: 1424)
Regression Formula: trip_time_minutes ~ trip_distance_units +
pickup_minute_of_day + C(day_of_week)

--- Regression Results Summary ---
                            OLS Regression Results

==============================================================================
========
Dep. Variable:      trip_time_minutes   R-squared:
0.998
Model:                            OLS   Adj. R-squared:
0.996
Method:                 Least Squares   F-statistic:
556.0
```

```
Date:                Wed, 26 Nov 2025   Prob (F-statistic):
4.55e-09
Time:                        01:53:11   Log-Likelihood:
0.29770
No. Observations:                  15   AIC:
15.40
Df Residuals:                       7   BIC:
21.07
Df Model:                           7

Covariance Type:            nonrobust

==============================================================================
====================
                             coef    std err          t      P>|t|
[0.025      0.975]
------------------------------------------------------------------------------
--------------------
Intercept                  1.5936      0.300      5.308      0.001
0.884       2.304
C(day_of_week)[T.Mon]     -0.7106      0.267     -2.660      0.032
-1.342      -0.079
C(day_of_week)[T.Sat]     -0.7826      0.279     -2.806      0.026
-1.442      -0.123
C(day_of_week)[T.Sun]     -0.8922      0.313     -2.848      0.025
-1.633      -0.151
C(day_of_week)[T.Tue]     -0.9316      0.309     -3.012      0.020
-1.663      -0.200
C(day_of_week)[T.Wed]     -0.3766      0.413     -0.911      0.392
-1.354       0.601
trip_distance_units        1.5962      0.032     50.325      0.000
1.521       1.671
pickup_minute_of_day       0.0050      0.000     21.299      0.000
0.004       0.006
==============================================================================
========
Omnibus:                        1.315   Durbin-Watson:
2.547
Prob(Omnibus):                  0.518   Jarque-Bera (JB):
0.278
Skew:                           0.307   Prob(JB):
0.870
Kurtosis:                       3.259   Cond. No.
4.54e+03
==============================================================================
========

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is
correctly specified.
```

```
[2] The condition number is large, 4.54e+03. This might indicate that
there are
strong multicollinearity or other numerical problems.

============================================================
--- Interpretation of Regression Results ---
============================================================
## 1. Overall Model Significance (F-Test)
----------------------------------
**Conclusion:** The overall model is **statistically significant** (p-
value: 0.0000).
This means the group of tested factors (distance, time of day, and day
of week) is useful in predicting trip duration.
**R-squared:** 0.9982. This indicates that 99.82% of the variance in
trip time is explained by the factors in this model.

************************************************************
## 2. Hypothesis Test for Trip Distance (Distance)
----------------------------------
Null Hypothesis (H0): Distance has no effect on trip time.
**Result:** Reject H0. The effect of distance is **statistically
significant** (p-value: 0.0000).
**Effect (Coefficient):** 1.5962
Interpretation: For every 1-unit increase in distance, the trip time
increases by approximately **1.60 minutes**, holding all other factors
constant.

************************************************************
## 3. Hypothesis Test for Time of Pickup (Minute of Day)
----------------------------------
Null Hypothesis (H0): Time of day has no effect on trip time.
**Result:** Reject H0. The effect of time of day is **statistically
significant** (p-value: 0.0000).
**Effect (Coefficient):** 0.005029
Interpretation: The time of day significantly affects trip time. The
positive coefficient suggests that later times in the day (higher
minute count) are associated with **slightly longer trip times** (or
that the inherent speed in the simulation varies by time).

************************************************************
## 4. Hypothesis Test for Day of Week
----------------------------------
Null Hypothesis (H0): The day of the week has no effect on trip time
(all day coefficients are zero).
**Result:** Reject H0. At least one day is **statistically
significant**.
Days that show a statistically different trip time (compared to the
baseline day, which is excluded by the model):
  - Mon: **Significant** (p=0.0325, Coef=-0.7106)
  - Sat: **Significant** (p=0.0263, Coef=-0.7826)
```

```
  - Sun: **Significant** (p=0.0248, Coef=-0.8922)
  - Tue: **Significant** (p=0.0196, Coef=-0.9316)

Interpretation: The trip time on these specific days is significantly
different from the baseline day, suggesting **varying levels of
congestion/speed limit effects** depending on the day.
============================================================
```

Creating the naive baseline model

- Uses time = distance by speed

Performance Metrics

- Mean Absolute Error
- Business Metric/Theshold: The ETA predicted should not be off by more than 5 minutes

```
SPEED_CONVERSION_FACTOR = 60

ride_df['predicted_time_naive'] = (
    ride_df['trip_distance_units'] / (ride_df['zone_speed_limit'] /
SPEED_CONVERSION_FACTOR)
)

ride_df['abs_error_naive'] = abs(ride_df['predicted_time_naive'] -
ride_df['trip_time_minutes'])

mae_naive = ride_df['abs_error_naive'].mean()

BUSINESS_THRESHOLD_MINUTES = 5

good_predictions_count = (ride_df['abs_error_naive'] <=
BUSINESS_THRESHOLD_MINUTES).sum()
total_predictions = len(ride_df)
business_success_rate = (good_predictions_count / total_predictions) *
100

print("\n" + "="*50)
print("Naive Model Performance Benchmarks")
print("="*50)

print(f"Prediction Formula: Time = Distance / (Speed Limit /
{SPEED_CONVERSION_FACTOR})")
print(f"Total Observations: {total_predictions}")
print("-" * 50)

print(f"1. Mean Absolute Error (MAE): {mae_naive:.2f} minutes")
print("\t(Average prediction is off by this many minutes.)")

print("-" * 50)
```

```
print(f"2. Business Threshold: Error <= {BUSINESS_THRESHOLD_MINUTES}
minutes")
print(f"\tGood Predictions Count: {good_predictions_count} /
{total_predictions}")
print(f"\tSuccess Rate: {business_success_rate:.2f}%")
print("=" * 50)

print("\nSample of Naive Predictions vs. Actual Time:")
print(ride_df[['trip_time_minutes', 'trip_distance_units',
'zone_speed_limit', 'predicted_time_naive',
'abs_error_naive']].head())
```

```
==================================================
Naive Model Performance Benchmarks
==================================================
Prediction Formula: Time = Distance / (Speed Limit / 60)
Total Observations: 15
--------------------------------------------------
1. Mean Absolute Error (MAE): 8.21 minutes
     (Average prediction is off by this many minutes.)
--------------------------------------------------
2. Business Threshold: Error <= 5 minutes
     Good Predictions Count: 4 / 15
     Success Rate: 26.67%
==================================================

Sample of Naive Predictions vs. Actual Time:
   trip_time_minutes  trip_distance_units  zone_speed_limit  \
0               9.76                    3              45.0
1              12.85                    6              45.0
2              21.01                   10              45.0
3              12.48                    4              20.0
4              15.90                    8              20.0

   predicted_time_naive  abs_error_naive
0              4.000000         5.760000
1              8.000000         4.850000
2             13.333333         7.676667
3             12.000000         0.480000
4             24.000000         8.100000
```

Feature Engineering: Identifying features and formatting before model use

```
ride_df['pickup_time_dt'] = pd.to_datetime(ride_df['pickup_time'],
format='%H:%M', errors='coerce')
ride_df['pickup_minute_of_day'] = ride_df['pickup_time_dt'].dt.hour *
60 + ride_df['pickup_time_dt'].dt.minute
```

```
target = 'trip_time_minutes'
numerical_features = ['trip_distance_units', 'zone_speed_limit',
'pickup_minute_of_day']
categorical_features = ['day_of_week', 'pickup_zone', 'dropoff_zone']
```

- Splitting Data on chronological order
- Performing one-hot encoding for categorical variables
- Performing standard feature scaling

Note:

- Train-Test split is done before encoding and scaling techniques
- One-hot encoding and festure scaling is fitted only to the training set and then transformed on both training and test datasets

```
ride_df['ride_date'] = pd.to_datetime(ride_df['ride_date'])
ride_df['request_minute_of_day'] = ride_df['request_time'].apply(
    lambda x: int(x.split(':')[0]) * 60 + int(x.split(':')[1])
)

ride_df = ride_df.sort_values(by='ride_date').reset_index(drop=True)

FEATURES = [
    'request_minute_of_day', 'day_of_week', 'trip_distance_units',
    'zone_speed_limit', 'pickup_zone', 'dropoff_zone'
]
TARGET = 'trip_time_minutes'

split_point = int(len(ride_df) * 0.8)
train_df = ride_df.iloc[:split_point]
test_df = ride_df.iloc[split_point:]

X_train = train_df[FEATURES].copy()
y_train = train_df[TARGET].copy()
X_test = test_df[FEATURES].copy()
y_test = test_df[TARGET].copy()

print(f"Total observations: {len(ride_df)}")
print(f"Train set size: {len(X_train)} (First date:
{train_df['ride_date'].min().date()}, Last date:
{train_df['ride_date'].max().date()})")
print(f"Test set size: {len(X_test)} (First date:
{test_df['ride_date'].min().date()}, Last date:
{test_df['ride_date'].max().date()})")

categorical_cols = ['day_of_week', 'pickup_zone', 'dropoff_zone']

X_train_encoded = pd.get_dummies(X_train, columns=categorical_cols,
drop_first=True)
```

```
X_test_encoded = pd.get_dummies(X_test, columns=categorical_cols,
drop_first=True)

X_test_encoded =
X_test_encoded.reindex(columns=X_train_encoded.columns, fill_value=0)

print("\n--- X_train_encoded.head() ---")
print(X_train_encoded.head().to_markdown(index=False, numalign="left",
stralign="left"))
print(f"\nX_train_encoded shape: {X_train_encoded.shape}")
print(f"X_test_encoded shape: {X_test_encoded.shape}")

from sklearn.preprocessing import StandardScaler
numerical_cols = ['request_minute_of_day', 'trip_distance_units',
'zone_speed_limit']

all_numerical_cols = X_train_encoded.select_dtypes(include=['int64',
'float64', 'uint8', 'bool']).columns.tolist()

cols_to_scale = [col for col in all_numerical_cols if col in
numerical_cols]

scaler = StandardScaler()

X_train_encoded[cols_to_scale] =
scaler.fit_transform(X_train_encoded[cols_to_scale])
X_test_encoded[cols_to_scale] =
scaler.transform(X_test_encoded[cols_to_scale])

print("\n--- X_train_encoded (After Scaling).head() ---")
print(X_train_encoded.head().to_markdown(index=False, numalign="left",
stralign="left"))
```

```
Total observations: 15
Train set size: 12 (First date: 2025-11-01, Last date: 2025-11-07)
Test set size: 3 (First date: 2025-11-07, Last date: 2025-11-07)

--- X_train_encoded.head() ---
| request_minute_of_day   | trip_distance_units   | zone_speed_limit
| day_of_week_Mon   | day_of_week_Sat   | day_of_week_Sun   |
day_of_week_Tue   | day_of_week_Wed   | pickup_zone_Park   |
pickup_zone_Road   | dropoff_zone_Road   |
|:------------------------|:----------------------|:-----------------
-|:------------------|:------------------|:------------------|:-------
-----------|:-----------------|:-------------------|:----------------
---|:--------------------|
| 58                      | 8                     | 20
| False             | True              | False             | False
| False             | False              | False              | True
```

```
|
| 1419                    | 12                    | 20
| False             | True              | False             | False
| False             | False              | False              | True
|
| 126                    | 9                    | 20
| False             | True              | False             | False
| False             | False              | False              | True
|
| 445                    | 8                    | 20
| False             | False             | True              | False
| False             | False              | False              | True
|
| 1281                    | 10                    | 45
| False             | False             | True              | False
| False             | False              | True              | True
|

X_train_encoded shape: (12, 11)
X_test_encoded shape: (3, 11)

--- X_train_encoded (After Scaling).head() ---
| request_minute_of_day   | trip_distance_units   | zone_speed_limit
| day_of_week_Mon   | day_of_week_Sat   | day_of_week_Sun   |
day_of_week_Tue   | day_of_week_Wed   | pickup_zone_Park   |
pickup_zone_Road   | dropoff_zone_Road   |
|:-----------------------|:----------------------|:-----------------
-|:-----------------|:-----------------|:-----------------|:-------
----------|:-----------------|:-------------------|:----------------
---|:--------------------|
| -1.63268                | 0.179369              | -0.720511
| False             | True              | False             | False
| False             | False              | False              | True
|
| 1.81418                | 1.40933              | -0.720511
| False             | True              | False             | False
| False             | False              | False              | True
|
| -1.46046                | 0.48686              | -0.720511
| False             | True              | False             | False
| False             | False              | False              | True
|
| -0.652564                | 0.179369              | -0.720511
| False             | False             | True              | False
| False             | False              | False              | True
|
| 1.46468                | 0.79435              | 1.15908
| False             | False             | True              | False
```

```
| False              | False                | True                 | True
|
```

First Model: The XGBoost Model

```
xgb_model = xgb.XGBRegressor(
    objective='reg:squarederror',
    n_estimators=100,
    learning_rate=0.05,
    max_depth=3,
    random_state=42,
    n_jobs=1,
    subsample=0.8,
    colsample_bytree=0.8
)

print("Training XGBoost Regressor...")
xgb_model.fit(X_train_encoded, y_train)
print("Training complete.")

y_pred_test = xgb_model.predict(X_test_encoded)

xgb_mae = mean_absolute_error(y_test, y_pred_test)
xgb_rmse = np.sqrt(mean_squared_error(y_test, y_pred_test))

xgb_test_results_df = pd.DataFrame({
    'Actual_Trip_Time': y_test,
    'Predicted_Trip_Time': y_pred_test,
    'Model': 'XGBoost'
}).reset_index(drop=True)

print("\n--- Test Set Evaluation ---")
print(f"Mean Absolute Error (MAE): {xgb_mae:.4f} minutes")
print(f"Root Mean Squared Error (RMSE): {xgb_rmse:.4f} minutes")
print("\n--- Test Set Predictions vs. Actual (First 5 Rows) ---")
print(xgb_test_results_df.head().to_markdown(index=False,
numalign="left", stralign="left"))
```

```
Training XGBoost Regressor...
Training complete.

--- Test Set Evaluation ---
Mean Absolute Error (MAE): 1.6051 minutes
Root Mean Squared Error (RMSE): 2.0268 minutes

--- Test Set Predictions vs. Actual (First 5 Rows) ---
| Actual_Trip_Time   | Predicted_Trip_Time   | Model   |
|:-------------------|:----------------------|:--------|
| 12.49              | 11.2546               | XGBoost |
```

```
| 14.49                | 14.1814               | XGBoost |
| 22.78                | 19.5086               | XGBoost |
```

Second Model: Lasso Regression

```
numerical_cols = ['request_minute_of_day', 'trip_distance_units',
'zone_speed_limit']

print("--- Feature Scaling (Required for Lasso) ---")

scaler = StandardScaler()
X_train_encoded[numerical_cols] =
scaler.fit_transform(X_train_encoded[numerical_cols])
X_test_encoded[numerical_cols] =
scaler.transform(X_test_encoded[numerical_cols])

print("Numerical features scaled.")
print("\n--- Training Lasso Regression Model ---")
LASSO_ALPHA = 0.1
lasso_model = Lasso(alpha=LASSO_ALPHA, random_state=42)

print(f"Training Lasso Regressor with alpha={lasso_model.alpha}...")
lasso_model.fit(X_train_encoded, y_train)
print("Training complete.")

y_pred = lasso_model.predict(X_test_encoded)

y_pred_series = pd.Series(y_pred, index=y_test.index,
name='Predicted_Trip_Time')

print("\n--- Test Set Evaluation ---")

lasso_mae = mean_absolute_error(y_test, y_pred)
lasso_rmse = np.sqrt(mean_squared_error(y_test, y_pred))

print(f"Mean Absolute Error (MAE): {lasso_mae:.4f} minutes")
print(f"Root Mean Squared Error (RMSE): {lasso_rmse:.4f} minutes")

lasso_test_results_df = pd.DataFrame({
    'Actual_Trip_Time': y_test,
    'Predicted_Trip_Time': y_pred_series,
    'Model': 'Lasso Regression'
}).reset_index(drop=True)

print("\n--- Test Set Predictions vs. Actual (First 5 Rows) ---")
print(lasso_test_results_df.head().to_markdown(index=False,
numalign="left", stralign="left"))

print("\n--- Lasso Model Coefficients (Top 5 Non-Zero) ---")
```

```
coef_df = pd.DataFrame({
    'Feature': X_train_encoded.columns,
    'Coefficient': lasso_model.coef_
}).sort_values(by='Coefficient', key=abs, ascending=False)

non_zero_coefs = coef_df[coef_df['Coefficient'].abs() > 1e-6]

if not non_zero_coefs.empty:
    print(non_zero_coefs.head(5).to_markdown(index=False,
numalign="left", stralign="left"))
else:
    print("All coefficients were shrunk to zero (alpha may be too
high).")
```

```
--- Feature Scaling (Required for Lasso) ---
Numerical features scaled.

--- Training Lasso Regression Model ---
Training Lasso Regressor with alpha=0.1...
Training complete.

--- Test Set Evaluation ---
Mean Absolute Error (MAE): 0.8760 minutes
Root Mean Squared Error (RMSE): 0.9389 minutes

--- Test Set Predictions vs. Actual (First 5 Rows) ---
| Actual_Trip_Time   | Predicted_Trip_Time   | Model            |
|:-------------------|:----------------------|:-----------------|
| 12.49              | 11.1462               | Lasso Regression |
| 14.49              | 13.9324               | Lasso Regression |
| 22.78              | 22.0535               | Lasso Regression |

--- Lasso Model Coefficients (Top 5 Non-Zero) ---
| Feature               | Coefficient   |
|:----------------------|:--------------|
| trip_distance_units   | 5.11602       |
| request_minute_of_day | 1.82995       |
```

```
# Export Metrics as a Dataset
metrics_data = {
    'Model': ['Lasso Regression', 'XGBoost'],
    'MAE (minutes)': [lasso_mae, xgb_mae],
    'RMSE (minutes)': [lasso_rmse, xgb_rmse]
}

metrics_df = pd.DataFrame(metrics_data)

metrics_df.to_csv('model_metrics.csv')
print("File exported: 'model_metrics.csv'")

# Export predictions as a dataset
```

```
combined_predictions_df = pd.concat([lasso_test_results_df,
xgb_test_results_df], ignore_index=True)
combined_predictions_df.to_csv('model_predictions.csv', index=False)
print("File exported: 'model_predictions.csv'")

#Export input data
dataset.to_csv('ride_dataset.csv', index=False)
print("File exported: 'ride_dataset.csv'")
```

```
File exported: 'model_metrics.csv'
File exported: 'model_predictions.csv'
File exported: 'ride_dataset.csv'
```